PaceVLA: 분석 리포트

게시용 버전: 논문 그림은 라이선스(arXiv 비독점 배포) 때문에 제외했습니다. 원문 https://arxiv.org/abs/2610.25961 · arXiv:2610.25961v2 · KAIST, RLWRLD, 고려대, 서울대 (Jeon, Lee, Kim, Oh, Kim, Koo, Koo, Ahn, Shin) · 코드 미공개 · 분석일 2026-10-29 · 논문 그림은 개인 학습용으로만 사용(논문 라이선스가 CC BY 아님)

핵심 요약

1. **무엇을 하는가:** VLA가 텔레옵 시연의 느린 속도를 물려받는 문제를, 이웃 행동을 합치는 시간 양자화(q = 2)를 국면에 따라 켜고 끄는 방식으로 푼다. GR00T N1.5의 공유 action expert에 모드별 decoder 4개를 달고, LVLM이 단 quantizability 점수로 학습한 judge와 라우터가 매 추론마다 모드를 고른다.
2. **표 내부는 정확히 맞는다:** ES를 SR·TPR·t_max만으로 다시 계산하면 36칸이 ±0.005 안에서 재현되고, Table 5의 24개 평균은 PnP 8 + Others 16 가중 평균과 같다. 실물 수치 주장, Fig. 1(b)의 다섯 점(6개 과제 단순 평균)도 모두 맞는다.
3. **우위는 "덜 실패하면서 충분히 빠르다"에서 나온다:** t_max = 1500이면 ES의 대부분이 성공률로 정해진다(실패 몫이 기준 T의 55-78%). 성공 시행만 보면 PnP에서 고정 양자화·DemoSpeedUp·SAIL이 모두 더 빠르고, TPR 1위인 칸은 8칸 중 2칸이다. 다만 실패를 14-115 step만 쳐도 ES 1위는 PaceVLA다.
4. **기준 대비 성공률 차이는 표본 오차 안이다:** 과제당 50회로 역산하면 p = 0.48-0.87이다. "유지"는 맞고 "향상"은 아니다.
5. **재현 정보가 빠져 있다:** judge 임계값 δ, 체크 가중치, 과제별 제어 이득을 고른 절차가 없다. 판정 기준은 평가 과제에서 고정 양자화를 돌려 본 결과를 단서로 Claude가 다듬었다. toy에서는 제어 이득만으로 고정 양자화의 성패가 갈렸다.
6. **구현 가능성:** 코드·라벨·체크포인트가 없어 그대로는 재현할 수 없다. 골격(모드별 decoder + 라우터 + judge)은 짧지만, 라벨 파이프라인(과제별 사전 실행 + LVLM 전체 라벨링)과 GR00T 미세조정(CUDA GPU 40 GB 이상 권장)이 장벽이다. 이 맥에서는 toy와 재계산까지 했고, 작은 정책 실험은 torch(MPS) 설치가 필요하다.

01. 논문 분석: PaceVLA (arXiv:2610.25961)

- 제목: *PaceVLA: Adaptive Temporal Action Quantization in Vision-Language-Action Models*
- 저자: Hojin Jeon*, Jimin Lee*, Taekwan Kim, Nayoung Oh, Taeyoung Kim, Myungkyu Koo, Jinmo Koo, Joonwoo Ahn, Jinwoo Shin (KAIST, RLWRLD, 고려대, 서울대)
- 공개: arXiv v1 2026-10-19, v2 2026-10-21 · 프로젝트 https://pacevla.github.io · **코드 미공개**(2026-10-29 확인, §02 4절)
- 라이선스: arXiv 비독점 배포(nonexclusive-distrib 1.0)라 **CC BY가 아닙니다.** 논문 그림은 개인용 리포트와 PPT에만 넣습니다.
- 분석 범위: v2 본문 9쪽과 부록 A-E(총 25쪽), 프로젝트 페이지, 관련 공개 코드(SAIL의 지표 함수, GR00T 저장소의 RoboCasa 과제 표)
- **v1 ↔ v2 차이:** 본문 텍스트는 단어 단위로 같습니다(머리글의 판 번호와 날짜만 다름). 바뀐 것은 Fig. 2의 화살표 하나입니다. v1에서는 VLM 특징이 첫 DiT 블록의 Cross-Attention과 Self-Attention으로 들어가게 그려져 있었습니다. v2에서는 두 블록의 Cross-Attention으로 각각 들어가게 고쳤습니다. 조건 경로를 바로잡은 그림 수정입니다.

---

1. Abstract (한 줄 요약과 주장)

한 줄 요약: 텔레옵 시연을 그대로 따라 하는 VLA는 사람이 조심스럽게 움직인 속도까지 물려받습니다. PaceVLA는 이웃한 행동을 합쳐 명령 수를 줄이는 **시간 양자화(temporal action quantization)**를, 정밀한 구간에서는 끄고 여유 있는 구간에서만 켭니다. 이를 위해 GR00T N1.5의 action expert에 모드별 출력 decoder 네 개를 달고, LVLM이 단 "양자화해도 되는가" 점수로 학습한 판정기(judge)와 라우터가 매 추론 step마다 모드를 고릅니다.

논문이 내세우는 기여는 세 가지입니다(Sec. 1).

1. 텔레옵 시연에서 물려받은 세밀한 실행 패턴을 VLA 비효율의 주요 원인으로 짚었습니다.
2. 모드별 decoder, quantization judge, context 조건 라우터로 된 가벼운 구조를 제안했습니다.
3. 시뮬레이션 벤치마크 3종과 실물 과제 6개에서, 기준 VLA와 고정 양자화보다 성공률과 효율의 균형이 낫다고 보고합니다. 기준 대비 최대 1.8배 빠르고, ALLEX 컨베이어 과제에서는 처리량이 1.75배입니다.

---

2. Method

2.1 전체 구조 (Fig. 2)

```
관측(카메라, 언어, 고유수용 상태) → VLM(고정) → 조건 특징 Φ_t
        │
        ├─ 평균 풀링 → quantization judge J_ψ → 점수 ≥ δ 이면 "양자화 그룹", 아니면 "세밀 그룹"
        │                  router (MLP)        → 그룹 안에서 모드 k 선택 (α_t)
        ▼
공유 action expert (DiT, flow matching) — 노이즈 행동 길이 T_k
        ▼
모드별 MLP decoder D_k → T_k개 명령 (H_k개 시연 step을 덮음, q_k = H_k / T_k)
```

모드는 K = 4개입니다(부록 C.3, Table 8·9).

	세밀 모드 (q = 1)	양자화 모드
RoboCasa, DexJoCo, Franka, OpenArm	(16, 16), (8, 8)	(16, 8), (8, 4) → q = 2
LIBERO	(16, 16), (8, 8)	(16, 9), (8, 5) → q = 1.78, 1.6
ALLEX	(24, 24), (12, 12)	(24, 12), (12, 6) → q = 2

따라서 "적응형 시간 해상도"는 연속 값이 아니라 **q ∈ {1, 2}** 중 하나를 고르는 이진 선택에, 예측 구간 두 가지를 곱한 것입니다. 명령 수로 따진 이론 최대 속도는 2배입니다.

2.2 시간 양자화 규칙 (Sec. 3.1, 부록 A)

- H개 시연 행동을 거의 같은 크기의 T개 연속 블록으로 나누고, 블록마다 명령 하나를 만듭니다.
- **절대 행동**(DexJoCo, 실물 관절 위치): 블록의 마지막 명령만 남깁니다(식 5). 끝점은 보존되고 중간 경유점은 사라집니다.
- **delta 자세 행동**(RoboCasa, LIBERO): 이동은 고정 좌표계에서 더하고, 회전은 SO(3)에서 합성하고(식 6), 그리퍼는 마지막 값을 씁니다. 정규화된 행동은 제어기 좌표로 되돌려 합친 뒤 다시 정규화합니다.

2.3 공유 action expert + 모드별 decoder (Sec. 3.2)

action encoder와 DiT는 모든 모드가 공유하고, 모드마다 출력 MLP decoder D_k만 따로 둡니다. 모드 k는 길이 T_k의 노이즈 행동 열을 처리합니다. 학습 때는 K개 모드의 flow matching 손실(식 1)을 모두 계산합니다. 모드마다 노이즈와 flow 시간을 따로 뽑고, 길이가 다른 목표는 마스크로 평균합니다.

2.4 Quantizability 지도와 모드 선택 (Sec. 3.3, 부록 C.5, E)

- **그룹 선택:** judge가 낸 점수가 임계값 δ 이상이면 양자화 그룹, 아니면 세밀 그룹을 씁니다. **δ의 값은 논문에 없습니다.**
- **모드 선택:** 라우터 확률 α_t가 그룹 안의 모드를 고릅니다.
- **점수 라벨 만들기:**
  1. 과제마다 고정 양자화를 걸어 실행해 보고, 과제별 성공률을 단서로 씁니다.
  2. 그 단서와 시연 예시로 Claude Opus 5가 벤치마크별 판정 기준과 프롬프트를 만듭니다. 기준은 4-6개의 시각 체크로 쪼개고, 각각 1-5점으로 매깁니다.
  3. Gemini 3.8 Flash가 데이터셋 전체의 chunk마다 첫 프레임 영상, 지시문, 시연에서 계산한 사실을 보고 체크별 점수를 답합니다.
  4. 고정 가중치로 합쳐 [0, 1]로 정규화한 것이 라벨 y입니다. **가중치와 "계산한 사실"의 내용은 공개되지 않았습니다.** 부록 E에 벤치마크별 프롬프트 6개가 원문으로 실려 있습니다.

2.5 학습 목표 (Sec. 3.4, 식 2-4)

```
L_policy = E[ Σ_k w_k ℓ_k + λ_sup KL(r ‖ α) + λ_bal L_bal ],   r_k ∝ exp(−ℓ_k / β)      (식 2)
L_conf   = E[ (J_ψ(sg(Φ_t)) − y)² ]                                                    (식 3)
L        = L_policy + λ_conf L_conf                                                     (식 4)
```

- 라우터는 "예측 오차가 작은 모드"를 목표 분포 r로 삼아 KL로 학습합니다. 학습 가중치 w는 α와 균등 분포를 섞은 것이고, L_bal은 decoder 하나로 쏠리는 것을 막습니다.
- judge는 VLM 특징에 stop-gradient를 걸고 라벨 y를 회귀합니다.
- 설정값(Table 7): λ_sup 0.1, λ_bal 0.05, λ_conf 0.1(앞의 두 값은 5k iteration 동안 0에서 선형 증가), w 온도 0.5, r 온도 0.3, 최소 모드 확률 0.05, AdamW lr 1e-4, 60k iteration, VLM 고정.

2.6 제어기 설정 (Sec. 3.5, 부록 C.3·C.4)

양자화된 명령은 한 주기에 더 큰 변화를 요구하므로 제어기 이득을 올립니다.

- **시뮬레이션:** OSC 위치 이득 k_p를 과제마다 {150, 300, 600} 중에서 고르고, **모든 가속 방법에 같은 과제별 이득**을 씁니다. 기준 GR00T N1.5만 기본 이득(150)을 씁니다. 이득을 어떤 절차로 골랐는지는 적혀 있지 않습니다.
- **실물:** Franka는 Cartesian 임피던스 강성 600 N/m(쌓기는 800), 그리퍼는 최대 속도와 최소 힘을 씁니다. OpenArm은 관절 PD 이득 $K_p \in \{240, 480\}$ Nm/rad입니다.

---

3. Experiments

3.1 설정 (Sec. 4.1, 부록 B·C)

- **시뮬레이션:**
  - RoboCasa-Kitchen 24개 과제(과제당 시연 300개, PnP와 Others로 나눠 보고)
  - LIBERO 4개 suite(Short = Spatial·Object·Goal 평균, Long)
  - DexJoCo 한 팔 과제 6개
  - 백본은 GR00T N1.5와 RLDX-1-IMG 두 가지입니다.
- **실물:**
  - Franka + 그리퍼: Pick-and-Place, Air Fryer, Stack Cup
  - OpenArm 양팔 + 6-DoF 손: Table Sweeping, Pump and Scrub
  - ALLEX 휴머노이드 + 15-DoF 손: Conveyor Package Handling
  - 시연 수는 과제마다 50-320개이고, 백본은 GR00T N1.5뿐입니다.
- **기준선:**
  - 고정 양자화(Tang et al. VLASH): 기준 모델 출력을 추론 시 q = 2로 합칩니다.
  - DemoSpeedup: 엔트로피 기준으로 시연을 다운샘플링합니다.
  - SAIL: 저자들이 재구현했습니다. end-effector delta로 바꾸고 action scheduling은 껐습니다(부록 C.6).
- **지표:**
  - **TPR** (SAIL을 따름, 식 7) = 평균(S_i/t_i – $(1–S_i)/t_{max}$). t_i는 실행한 행동 step 수입니다.
  - **ES** (식 8) = T_{base} / T, $T = L + (1–SR)/SR \cdot t_{max}$. L은 성공 시행의 평균 step이고, 실패 한 번에 t_{max}(RoboCasa·DexJoCo 1500, LIBERO 220-520)를 매깁니다.
  - **실물:** 성공률(부분 점수 0.5 포함, 32회), **완전 성공 시행만의 평균 시간**, ALLEX는 분당 처리 패키지 수(5회 × 10개)입니다.

3.2 시뮬레이션 결과 (Table 1, 3)

GR00T N1.5	RoboCasa PnP SR / TPR / ES	Others SR / TPR / ES	LIBERO Short	LIBERO Long	DexJoCo
기준	52.3 / 1.05 / 1.00	72.4 / 2.20 / 1.00	97.8 / 8.36 / 1.00	89.2 / 3.12 / 1.00	76.3 / 1.83 / 1.00
+ 고정 양자화	49.0 / 1.56 / 0.96	63.1 / 2.92 / 0.81	91.2 / **12.41** / 1.25	82.0 / **4.78** / 1.21	63.0 / 1.65 / 0.70
+ DemoSpeedUp	40.3 / 1.15 / 0.71	57.3 / 2.77 / 0.67	87.7 / 12.09 / 1.14	71.6 / 3.91 / 0.90	65.7 / 1.75 / 0.76
+ SAIL	32.5 / 0.77 / 0.52	58.5 / 2.31 / 0.68	95.5 / 10.85 / 1.23	79.0 / 3.89 / 1.03	60.3 / 1.58 / 0.65
+ PaceVLA	**55.0 / 1.64 / 1.16**	**72.9 / 3.30 / 1.14**	97.1 / 10.95 / **1.27**	89.2 / 4.27 / **1.26**	**77.3 / 2.22 / 1.11**

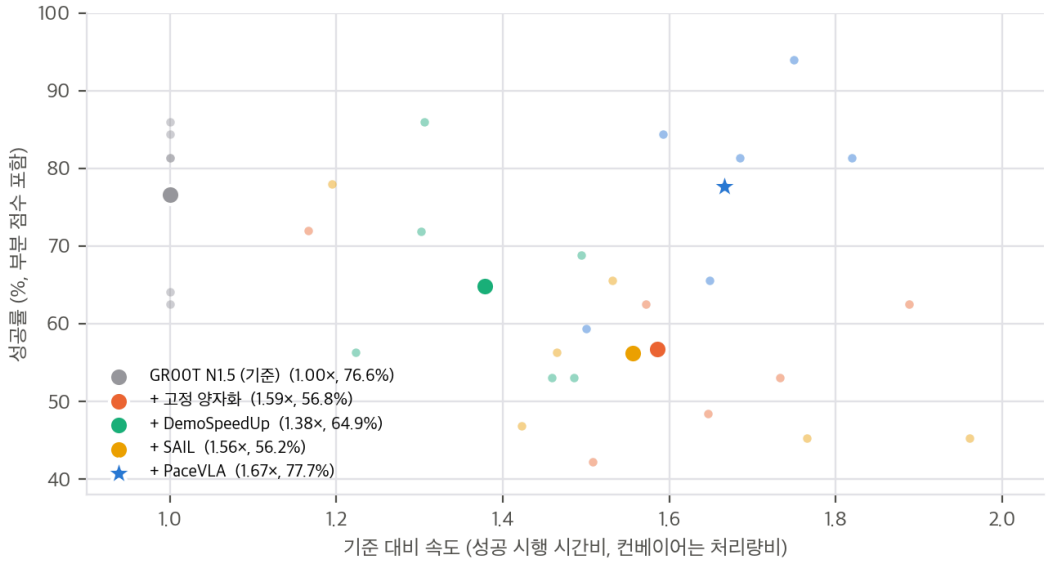
해석 (재계산은 `04_claim_verification.md` §3과 `analysis/table_check.py`):

- **표 내부는 정확히 맞습니다.** ES를 SR, TPR, t_max만으로 다시 계산하면 36칸이 모두 ±0.005 안에서 재현됩니다(성공 궤적 길이는 TPR에서 조화평균으로 복원). Table 5의 "24개 과제 평균"(66.9, 2.75)도 PnP 8개와 Others 16개의 가중 평균(66.93, 2.747)으로 정확히 나옵니다. 과제 구성은 GR00T 저장소의 RoboCasa 표에서 확인했습니다.
- **ES는 성공률이 지배합니다.** t_max = 1500이면 기준 모델의 "성공 1건당 step"에서 실패 몫이 PnP 78%, Others 65%, DexJoCo 55%입니다. 그래서 ES 순위는 속도보다 "성공률을 얼마나 지켰나"로 갈립니다.
- **성공한 시행만 보면 기준선이 더 빠릅니다.** t_max = 0(실패 비용 없음)일 때 PnP에서 고정 양자화 1.48, DemoSpeedUp 1.47, SAIL 1.44, PaceVLA 1.35입니다(`03` §1.2). PaceVLA의 장점은 "더 빠르다"가 아니라 "덜 실패하면서 충분히 빠르다"입니다. 다만 순위가 뒤집히는 t_max는 14-115 step이라, 실패를 조금만 비용으로 쳐도 PaceVLA가 ES 1위입니다.
- **TPR은 1위가 아닌 칸이 많습니다.** Table 1의 8칸 가운데 PaceVLA의 TPR이 가속 방법 중 1위인 칸은 2개뿐입니다. ES는 7칸, SR은 8칸에서 1위입니다. 굵게 표시된 칸만 보면 이 차이가 잘 드러나지 않습니다.
- **기준 대비 SR 차이는 표본 오차 안입니다.** 과제당 50회로 보면(시행 수 격자 검사로 확인) PnP 55.0 vs 52.3은 $p = 0.48$, Others 72.9 vs 72.4는 $p = 0.87$, DexJoCo 77.3 vs 76.3은 $p = 0.85$입니다. "성공률을 유지한다"는 주장과는 맞지만, "성공률도 오른다"고 읽으면 안 됩니다. 기준선 대비로는 유의합니다(Others에서 고정 양자화 대비 $p = 3.6\times10^{-5}$).

3.3 실물 결과 (Table 2, 4)

과제 (32회)	기준 SR / 시간	고정 양자화	PaceVLA SR / 시간	시간 감소
Pick-and-Place	81.3 / 12.1 s	62.5 / 7.7 s	84.4 / 7.6 s	37.2%
Air Fryer	81.3 / 24.6 s	53.1 / 14.2 s	81.3 / 14.6 s	40.7%
Stack Cup	84.4 / 35.3 s	62.5 / 18.7 s	81.3 / 19.4 s	45.0%
Table Sweeping	62.5 / 21.9 s	48.4 / 13.3 s	59.4 / 14.6 s	33.3%
Pump and Scrub	64.1 / 30.0 s	42.2 / 19.9 s	65.6 / 18.2 s	39.3%
ALLEX Conveyor (50개)	86% / 3.6개/분	72% / 4.2	94% / 6.3	처리량 1.75배

- 본문의 수치 주장(37-45%, 33-39%, 고정 양자화의 28.2·21.9%p 하락, 1.75배, 최대 1.8배)은 모두 표와 맞습니다.
- **Fig. 1(b)의 다섯 점은 6개 과제의 단순 평균입니다.** 그림을 400 dpi로 판독하면 성공률은 ±0.1%p, 속도는 +0.02× 안에서 평균값(PaceVLA 1.67×, 77.7%)과 맞습니다. 다섯 과제의 "완전 성공 시행만의 시간비"와 컨베이어의 "처리량비"를 섞어 평균한 값입니다. 축 이름은 "Speedup over demonstration"인데 캡션은 "over the base VLA"이고, 점선도 기준 VLA 위치를 "Demo speed"로 표시합니다.
- **PaceVLA는 고정 양자화와 거의 같은 속도입니다.** 시간비가 고정 양자화의 0.92-1.10배입니다. 명령 수가 시간에 비례한다고 단순화하면, 양자화 모드를 쓴 비율은 Pick-and-Place 74%에서 Stack Cup 90%로 추정됩니다(상한 추정). 세밀 모드로 돌아가는 구간이 짧다는 뜻입니다.
- **시행 수는 표기와 정합합니다.** 이진 과제의 성공률은 모두 k/32, 부분 점수 과제는 k/64, 컨베이어는 k/50 꼴입니다. Air Fryer에서 PaceVLA 대 고정 양자화(26 vs 17/32)는 $p = 0.03$이고, 기준 대비 차이는 모두 유의하지 않습니다.



작은 점: 과제 6개(Table 2·4), 큰 점: 단순 평균 = Fig. 1(b)의 점. 시간은 완전 성공 시행만의 평균이라 실패 비용이 빠져 있음.

우리 분석: 실물 6개 과제의 성공률 대 속도. 큰 점(6개 평균)이 Fig. 1(b)의 점이다.

3.4 Ablation (Table 5, RoboCasa 24개 평균, GR00T N1.5)

변형	SR	TPR	ES
모드별 decoder 없음(judge가 양자화를 고르면 기준 decoder 출력을 사후에 합침)	63.3	2.44	1.00
judge 없음(라우터가 네 모드 중 직접 선택)	65.1	2.01	1.01
KL 없음 / Balance 없음	63.7 / 64.8	2.04 / 2.38	0.98 / 1.04
Attention pooling / 마지막 토큰	64.6 / 64.2	2.38 / 2.43	1.04 / 1.03
PaceVLA	**66.9**	**2.75**	**1.14**

- 기준 GR00T N1.5의 24개 평균은 SR 65.7, TPR 1.82입니다(Table 1 가중 평균). 모든 변형이 ES 1.00 근처라서, ES 이득의 대부분은 "모드별 decoder + judge + KL"이 모두 있을 때만 생깁니다.
- 단일 시드입니다. 1,200회 기준으로도 decoder 제거의 SR 차이(66.9 vs 63.3)는 p = 0.07입니다.
- 부록 D.1(Table 10)은 전용 decoder가 사후 양자화보다 양자화 목표를 더 정확히 예측한다고 보입니다. 두 설정 평균 MSE가 0.0218 vs 0.0294입니다.

3.5 정성 분석과 비용

- **Fig. 4:** 한 RoboCasa 시연의 라벨 점수는 쥐기와 놓기에서 0.1-0.15로 낮고, 도달과 운반에서 0.44-0.56입니다. 그런데 점수가 **0.56을 넘지 않으므로**, δ가 0.5 근처라면 도달 구간의 일부(0.44)도 세밀 모드가 됩니다. δ를 모르면 이 그림만으로 실제 모드 선택을 알 수 없습니다.
- **Table 6:** 추론 지연은 RTX 5090에서 40.07 ms → 42.15 ms(1.05배, +2.1 ms)입니다.
- **부록 D.3:** 실패 사례 대부분이 인식 오류(약간 빗나간 파지, 손에 가려진 접촉 영역)라고 분석합니다.

---

4. Limitations

4.1 저자가 밝힌 한계

judge와 라우터 때문에 추론이 조금 느려집니다(1.05배). 실패는 대부분 인식 한계에서 오므로, 깊이나 촉각 같은 더 나은 감지가 필요하다고 봅니다(부록 D.3).

4.2 분석하면서 추가로 발견한 한계와 의문점

1. **판정 기준이 평가 결과를 보고 만들어졌습니다.** quantizability 기준은 "과제별로 고정 양자화를 걸어 본 성공률"을 단서로 Claude가 다듬은, 벤치마크별 수작업 프롬프트입니다(부록 C.5, E). 저자들은 성공률을 판정기의 학습 목표로 쓰지 않았다고 밝힙니다. 그래도 평가 과제의 결과가 라벨 설계에 들어갔고, 기준선에는 이런 과제별 조정이 없습니다. 새 과제에 적용하려면 이 반복을 다시 해야 합니다.
2. **재현에 필요한 값이 빠져 있습니다.** judge 임계값 δ, 체크별 가중치, 프롬프트에 넣는 "계산한 사실", 과제별 k_p를 고른 절차, 시뮬레이션 시행 수, 시드가 없습니다. 시행 수는 표 수치에서 과제당 50회로 역산했습니다.
3. **제어 이득이 교란 변수입니다.** 가속 방법들은 과제별로 조정한 이득을 쓰고, 기준 모델은 기본 이득을 씁니다. 우리 toy에서는 추적 지연만 있는 과제라면 이득을 kp 150에서 300으로 올리는 것만으로 고정 q = 2의 성공률이 0에서 0.985로 바뀌었습니다(`03` §2). 논문 과제에는 추적 지연 말고도 접촉 같은 실패 요인이 있으므로 toy를 그대로 옮길 수는 없습니다. 그래도 이득 조정 없이 같은 비교를 해야 기여를 분리할 수 있습니다.
4. **빠진 기준선이 있습니다.** "양자화 decoder를 학습시키고 항상 그것만 쓰기"가 없습니다. judge를 뺀 ablation은 라우터가 네 모드를 다 고르는 설정이라 이것과 다릅니다. 고정 양자화 기준선은 학습 없이 기준 모델 출력을 합친 것이라, 전용 decoder의 이득과 judge의 이득이 섞입니다.
5. **실물 시간은 완전 성공 시행만의 평균입니다.** 실패 비용이 빠져 있고, 성공한 시행만 고르는 선택 효과가 방법마다 다릅니다. 실물에는 시뮬레이션의 ES에 해당하는 지표가 없습니다(컨베이어 처리량은 예외).
6. **"적응형 해상도"의 범위가 좁습니다.** 실제 선택지는 $q \in \{1, 2\}$(LIBERO는 1.6, 1.78)와 예측 구간 두 가지입니다. 이론 최대 속도는 2배이고, 실물 속도 1.5-1.8배는 그 상한에 가깝습니다.
7. **기준선 구현이 원래와 다릅니다.** SAIL은 핵심 장치인 action scheduling을 끄고 delta 행동으로 바꿔 재구현했습니다. DemoSpeedup의 적용 방법은 따로 설명되지 않았습니다.
8. **컨베이어 과제 설명이 서로 맞지 않습니다.** 본문은 "도착한 패키지를 라벨이 앞을 보게 돌려 놓는다", 부록 C.1은 "컨베이어에서 집어 하역 테이블에 놓는다", 지시문은 "바코드를 위로 해서 컨베이어 위에 놓는다"입니다. 또 ALLEX만 비동기 추론과 real-time chunking을 씁니다(부록 C.6).
9. **코드와 라벨이 공개되지 않았고, 라벨링 비용도 없습니다.** Gemini API로 데이터셋 전체의 chunk마다 영상 2-3장씩 질의한 비용이 보고되지 않았습니다.

03. 지표 해부와 toy 검증

재계산 스크립트는 `analysis/table_check.py` (표·본문·Fig. 1(b), 결과 `table_check_results.json`)와 `analysis/quant_toy.py` (SO(3) 집계, 추적 toy, ES의 t_max 의존성, 결과 `quant_toy_results.json`)입니다. 모두 numpy·scipy 단일 스레드로 CPU에서 10초 안에 끝납니다.

1. TPR과 ES는 같은 시행 데이터의 두 요약이다

식 7과 8을 나란히 쓰면 다음과 같습니다(SR = s, 성공 시행의 step 수 t_i).

```
TPR = (1/N) Σ [ S_i / t_i − (1 − S_i) / t_max ] = s · E[1/t | 성공] − (1 − s)/t_max
ES  = T_base / T,   T = L + (1 − s)/s · t_max,   L = E[t | 성공]
```

첫 식에서 성공 궤적 길이의 **조화평균** L_h = s / (TPR + (1 − s)/t_max)를 복원할 수 있습니다. L 자리에 L_h를 넣어 ES를 다시 계산하면 Table 1·3의 36칸이 모두 ±0.005 안에서 맞습니다(LIBERO Short는 t_max 220·280·300의 평균으로 근사). 따라서 두 지표는 내부적으로 일관되고, 성공 시행의 길이 분산도 작다고 볼 수 있습니다.

1.1 ES는 무엇이 움직이나 (GR00T N1.5 + PaceVLA)

열	기준 L_h	PaceVLA L_h	기준 T 중 실패 몫	성공률만 바뀐 경우	속도만 바뀐 경우	둘 다(보고 ES)
RoboCasa PnP	382	284	**78%**	1.087	1.060	1.159 (1.16)
RoboCasa Others	304	209	65%	1.016	1.121	1.141 (1.14)
LIBERO Long	268	199	19%	1.000	1.263	1.263 (1.26)
DexJoCo	384	326	55%	1.031	1.073	(1.11)

- PnP에서는 성공률 +2.7%p(표본 오차 안, p = 0.48)가 ES 이득의 절반 이상을 만듭니다. Others와 LIBERO Long에서는 속도가 주된 몫입니다.
- 성공 시행만의 step 비(L_h 비)는 PnP 1.35, Others 1.45, Long 1.35, DexJoCo 1.18입니다. 이론 최대 2배와 실물 1.5-1.8배보다 작습니다.

1.2 t_max에 따른 순위

Table 1의 SR·TPR에서 L_h를 복원하고(원래 t_max = 1500으로), t_max만 바꿔 식 8을 다시 계산했습니다.

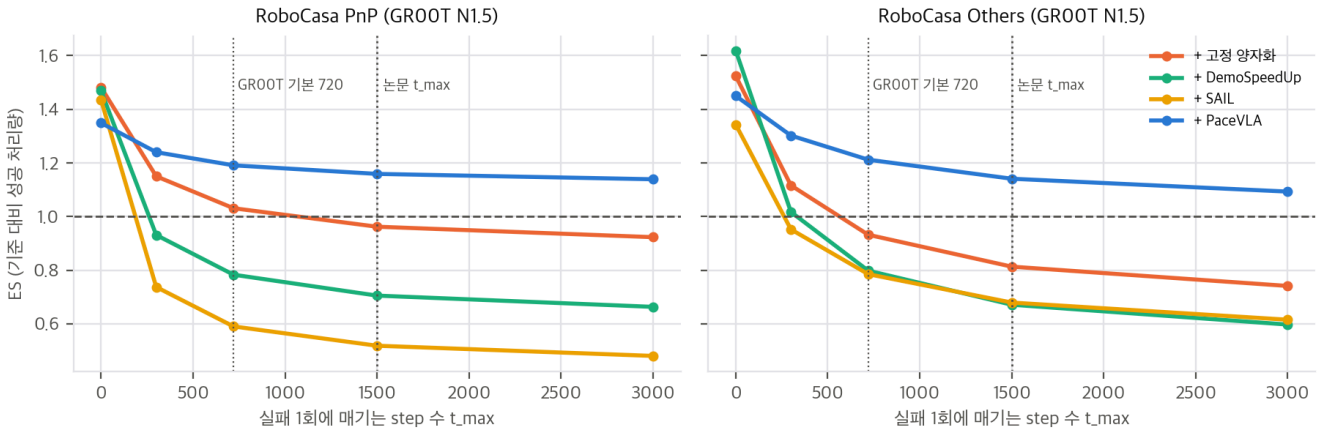


Table 1의 SR·TPR에서 성공 궤적 길이를 복원(조화평균)하고 t_max만 바꿔 식 8을 다시 계산. t_max = 0은 성공 시행의 step 비.

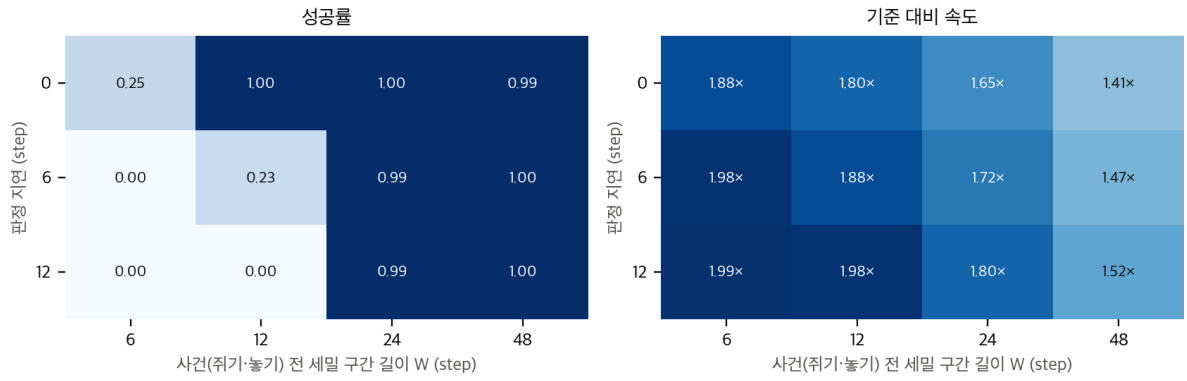
RoboCasa PnP	t_max 0	300	720	1500	3000
고정 양자화	**1.48**	1.15	1.03	0.96	0.92
DemoSpeedUp	1.47	0.93	0.78	0.71	0.66
SAIL	1.44	0.74	0.59	0.52	0.48
PaceVLA	1.35	**1.24**	**1.19**	**1.16**	**1.14**

- t_max = 0(실패 비용 없음, 성공 시행끼리의 속도)에서는 PnP의 세 기준선이 모두 PaceVLA보다 빠릅니다. Others에서는 고정 양자화(1.52)와 DemoSpeedUp(1.62)이 PaceVLA(1.45)보다 빠르고, SAIL(1.34)은 느립니다.
- 순위가 뒤집히는 t_max는 PnP 고정 양자화 약 115, DemoSpeedUp 35, SAIL 14이고, Others 고정 양자화 48, DemoSpeedUp 59입니다. 즉 실패 한 번을 에피소드 하나의 일부(PnP 기준 성공 궤적 약 380 step의 30%)만큼만 쳐도 PaceVLA가 1위입니다.
- 결론: **PaceVLA의 우위는 "실패를 덜 한다"에서 나옵니다.** 실패 비용을 크게 보는 운영 환경(실패하면 사람이 개입하고 다시 시작해야 하는 경우)일수록 이 판단이 맞습니다.

2. 추적 toy: 세밀 구간의 길이와 판정 시점

설정: 2차원 손끝이 60 Hz 시연 경로(곡선 도달 → 쥐기 → 운반 → 놓기, 240 step)를 따라갑니다.

- 절대 위치 명령은 블록 끝값(식 5)으로 양자화하고, 명령 하나가 제어 한 주기입니다.
- 손끝은 임계감쇠 2차 PD(이득 kp)로 명령을 추적합니다.
- 쥐는 순간과 놓는 순간의 위치 오차가 4 mm 안이면 성공입니다. 쥐는 step이 블록 안에 묻히면 가장 가까운 명령 시점에서 판정합니다.
- 시연에는 1.5 mm 떨림을 넣었고, 조건마다 200회 반복했습니다.
- 적응형은 "사건 전 W step부터 사건 뒤 4 step까지 세밀 모드"이고, 판정 지연 d만큼 세밀 구간 시작이 늦어집니다.



kp 150, q=2 · 고정 q=2는 성공률 0.00·속도 1.99×, kp 300 이상에서는 고정 q=2도 0.98. analysis/quant_toy.py

kp	기준 q=1	고정 q=2	적응형 W=12, d=0	W=12, d=6	W=24, d=12	W=48, d=0
150	0.995 / 1.00×	**0.00** / 1.99×	1.00 / 1.80×	**0.23** / 1.88×	0.995 / 1.80×	0.995 / 1.41×
300	0.975 / 1.00×	**0.985** / 1.99×	0.975 / 1.80×	0.99 / 1.88×	0.99 / 1.80×	0.995 / 1.41×

(성공률 / 기준 대비 명령 수 비)

해석:

- **추적 이득이 낮으면** 고정 양자화가 완전히 실패하고, 적응형만 성공합니다. 이때 세밀 구간이 짧을수록 빠르지만 판정 시점에 민감해집니다. W = 12에서 판정이 6 step(0.1초) 늦으면 성공률이 1.00에서 0.23으로 떨어집니다. 넉넉한 구간(W = 24)은 12 step 지연에도 버티지만 속도를 내줍니다.
- **이득을 두 배로 올리면** 이 toy에서는 고정 q = 2도 성공합니다. 추적 지연이 유일한 실패 원인이라면, 적응형의 이득은 제어기 설정에 강하게 의존합니다. 논문이 가속 방법마다 과제별 k_p를 따로 조정한 것(부록 C.3)이 비교에 영향을 줄 수 있는 이유입니다.
- 실제 과제에는 접촉 역학, 물체 미끄러짐, 시각 피드백 지연처럼 이득으로 해결되지 않는 정밀도 요구가 있습니다. 논문에서 이득을 조정한 뒤에도 고정 양자화의 성공률이 떨어지는 것(Air Fryer −28.2%p)은 그런 요인 때문으로 보입니다. 이 toy는 메커니즘 하나만 보여 주는 것입니다.

3. delta 회전을 합칠 때 SO(3) 합성이 필요한가 (부록 A, 식 6)

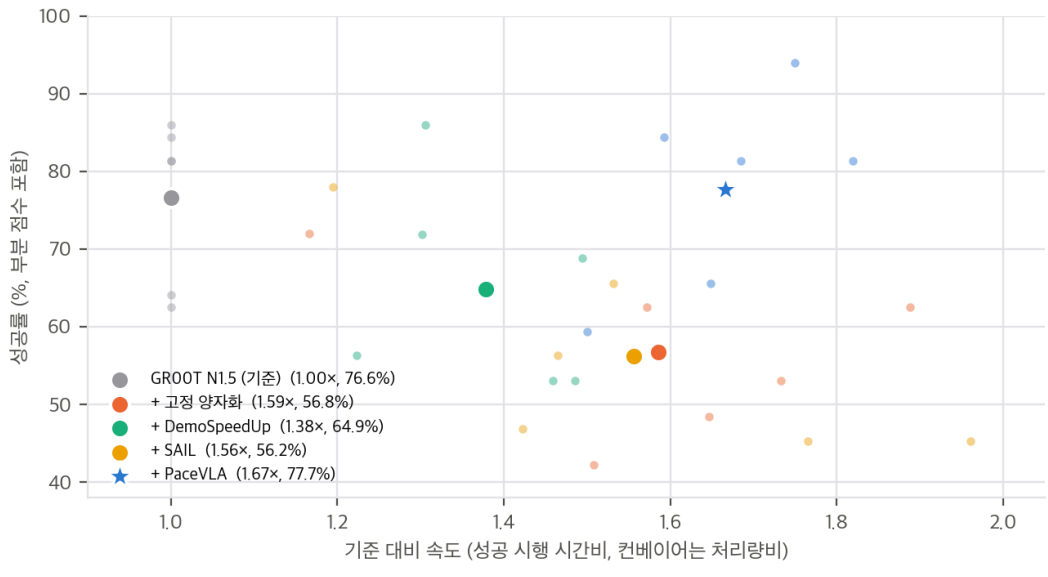
스텝마다 축이 무작위인 회전 증분(크기 θ) q개를 두 방식으로 합쳐, 결과 회전의 차이를 쟀습니다. 첫째는 회전 벡터를 그냥 더하는 방식, 둘째는 순서대로 곱하는 방식(공간 증분 규약)입니다. 조건마다 400회 반복했습니다.

q	θ = 0.01 rad	0.03 rad	0.1 rad
2	0.0025° (0.3%)	0.022° (0.9%)	0.25° (3.1%)
4	0.0046° (0.4%)	0.040° (1.3%)	0.46° (4.2%)
8	0.0097° (0.7%)	0.088° (2.1%)	0.98° (6.6%)

(오차 중앙값, 괄호는 합성 회전 크기 대비 비율)

- 오차는 대략 $\theta^2 \cdot q$에 비례합니다(교환자 항). q = 2이고 스텝당 회전이 수 도 이하이면 단순 합과 합성의 차이는 0.1° 미만입니다.
- 따라서 이 논문 설정(q = 2)에서 식 6은 "정확성을 위한 올바른 처리"이지만, 성능 차이를 만들 만한 요소는 아닙니다. q를 4-8로 키우거나 빠른 손목 회전을 합칠 때 중요해집니다.

4. 실물 결과의 구조



작은 점: 과제 6개(Table 2·4), 큰 점: 단순 평균 = Fig. 1(b)의 점. 시간은 완전 성공 시행만의 평균이라 실패 비용이 빠져 있음.

- **Fig. 1(b) = 6개 과제의 단순 평균**입니다. 성공률(부분 점수 포함)과 속도(다섯 과제는 완전 성공 시행의 시간비, 컨베이어는 처리량비)를 섞어 평균했습니다. 판독값과 재계산값의 차이는 성공률 ±0.1%p, 속도 +0.02×이고, 속도 쪽 편차는 다섯 점에서 같은 방향이라 판독 눈금 오차로 보입니다.
- **PaceVLA와 고정 양자화의 시간비가 0.92-1.10입니다.** PaceVLA가 세밀 모드로 돌아가는 구간이 짧다는 뜻입니다. 명령 수가 시간에 비례한다고 단순화하면 양자화 모드 비율은 Pick-and-Place 74%, Air Fryer 81%, Stack Cup 90%, Table Sweeping 67%, Pump and Scrub 79%입니다. 추론 지연과 안착 시간이 있으므로 실제 비율의 상한 추정으로 읽어야 합니다.
- **시간은 완전 성공 시행만 셉니다.** 성공률이 낮은 방법일수록 쉬운 초기 조건의 시행만 남아 시간이 짧게 측정될 수 있습니다. 실물에도 ES 같은 실패 비용 포함 지표가 있었다면 기준선들의 속도 이점은 더 줄었을 것입니다.

5. Table 5와 기준 모델의 24개 평균

- GR00T N1.5 + PaceVLA: Table 1에서 (8 × 55.0 + 16 × 72.9) / 24 = **66.93**, TPR (8 × 1.64 + 16 × 3.30) / 24 = **2.747**. Table 5의 66.9, 2.75와 같습니다. 같은 방식으로 풀링해 ES를 다시 계산하면 1.141로, Table 5의 1.14와 맞습니다.
- 기준 GR00T N1.5의 24개 평균은 SR 65.7, TPR 1.82입니다. Table 5의 모든 변형은 SR 63.3-66.9, ES 0.98-1.14 범위입니다. 모드별 decoder를 빼면 SR이 기준보다 낮아지고(63.3), judge를 빼면 TPR이 크게 줄어듭니다(2.01).
- 시행 수를 1,200회(24개 × 50)로 보면 full과 decoder 제거의 SR 차이는 p = 0.07입니다. 단일 시드라서 Table 5의 순서는 경향으로 읽는 것이 맞습니다.

추가 검증 (2026-11-11)

스크립트: `analysis/extra_quant_policy.py` → `analysis/extra_quant_policy_results.json` (PyTorch CPU, 4 스레드, 실행 171초). §2의 추적 toy를 학습 가능한 flow matching 정책으로 확장한 것입니다.

추가 검증 1: 같은 추적 이득에서 전용 양자화 decoder와 물리 판정 라우터의 기여

목적. 고정 양자화(사후 양자화), 학습된 양자화 decoder, 물리 판정 라우터를 같은 이득 kp에서 비교합니다. 이득을 바꾸는 교란 없이 전용 decoder와 판정기의 몫을 나눠 봅니다.

설정(가정).

- 환경: 2차원 손끝이 임계감쇠 PD(이득 kp ∈ {150, 300})로 명령을 따라갑니다. 시간 간격 1/60 s, 240 step. 시연은 시작 → 쥐는 점 g(80 step에 도달, 20 step 정지) → 놓는 점 r(180 step에 도달, 60 step 정지). 목표 g, r은 $[0.3, 0.7]^2$ m에서 임의로 뽑습니다.
- 명령 시연: 깨끗한 경로를 PD 역동역학으로 정확히 따라가는 명령을 만든 뒤 명령에 0.5 mm 잡음을 더했습니다. 처음에는 명령이 경로를 따라잡지 못해 기준 정책도 성공률 0이었습니다. 이 점은 아래 한계에 적습니다.
- 정책: 조건부 flow matching, 8개 명령 chunk(H = 8), 조건은 손끝 위치·속도, 목표 g·r, 구간 시작 자세, 시각. 정책은 손끝 위치 대비 명령 잔차를 생성합니다. 학습 데이터는 명령 잡음 2 cm를 더해 벗어난 상태를 포함시킨 시연 궤적 3000개에서 얻습니다(잡음 0.3 cm와 1 cm에서는 닫힌 루프가 무너졌습니다).
- 네 변형: base = fine 정책(q = 1). posthoc = fine 정책의 블록 끝값(원소 1)을 q = 2로 실행. decoder = q = 2 블록 끝값을 직접 학습한 정책. router = 목표까지 거리 < 3 cm이면 fine(q = 1), 아니면 decoder(q = 2). 판정은 물리 거리 규칙입니다.
- 성공: 80 step에서 손끝과 g의 거리 < 4 mm이고 180 step에서 손끝과 r의 거리 < 4 mm. 평가 300 episode, 시드 2개, 학습 8000 step.
- 장치: CPU. 패키지: torch 2.15.0, numpy 1.26.4, Python 3.12.7.

결과 (성공률, 시드 2개 평균; 괄호는 시드별 값; 명령 수 비는 q = 1 대비).

변형	kp = 150	kp = 300	명령 수 비
base (fine, q = 1)	0.38 (0.32, 0.43)	0.38 (0.51, 0.24)	1.00
posthoc (사후 양자화, q = 2)	0.30 (0.37, 0.23)	0.35 (0.48, 0.22)	0.50
decoder (학습된 양자화, q = 2)	0.58 (0.62, 0.53)	0.51 (0.58, 0.44)	0.50
router (물리 판정)	0.55 (0.53, 0.57)	0.53 (0.67, 0.38)	0.78

결론.

- 같은 이득에서 사후 양자화(q = 2로 실행)는 기준보다 나아지지 않았습니다(0.30-0.35 대 0.38). 시드 2개의 차이 안입니다.
- 학습된 양자화 decoder는 같은 명령 수(0.5)에서 사후 양자화보다 높습니다(0.58 대 0.30, 0.51 대 0.35). 이득을 바꾸지 않은 상태에서 "양자화된 명령을 학습하는 것"의 기여가 이 toy에서 가장 큰 항목입니다.
- 물리 판정 라우터는 decoder와 비슷하고(0.55 대 0.58, 0.53 대 0.51), 명령 수는 0.78로 줄었습니다. 즉, 판정기를 더하는 이득은 이 toy에서 보이지 않았습니다. decoder가 이미 정확한 명령을 주므로 판정기를 더해도 성공률이 오르지 않습니다.
- 기준 fine 정책이 decoder보다 낮은 것은 주의해서 읽어야 합니다. 기준은 명령을 매 step 새로 뽑는 정책이고, decoder는 블록 단위의 더 부드러운 목표를 학습했습니다. 이 차이가 "양자화가 도움이 된다"는 뜻은 아닙니다. 두 정책의 학습 목표 차이가 섞여 있습니다.

한계. 시드 2개이고 시드별 편차가 큽니다(base kp = 300: 0.24-0.51). 성공률의 차이는 대부분 0.1-0.2 범위입니다. 시연과 잡음, 상태 입력(구간 시작 자세 포함, 효과는 별도로 확인하지 않음)은 모두 가정입니다. 닫힌 루프 안정화에 실제로 필요했던 것은 명령 잡음(2 cm)이었습니다. 구간 시작 자세 입력은 잡음 0.3 cm에서 추가했을 때 오차를 줄이지 못했고, 설계에 남겨 두었을 뿐 효과는 확인하지 못했습니다. 이 점은 결과에 영향을 주지 않는다고 판단하지만, 엄밀한 제거 실험은 하지 않았습니다. 물리 판정은 실제 LVLM 판정기와 다릅니다. 실제 과제(접촉, 미끄러짐, 시각 지연)와 논문 Table의 수치는 이 toy에서 재현하지 않았습니다.

04. 주장 검증: 표·그림 재계산, 재현 정보, 구현 가능성

- **대상:** arXiv:2610.25961 v2(본문과 부록 A-E), 프로젝트 페이지(`PaceVLA/pacevla.github.io` @ `86424ec`)
- **코드:** 공식 구현이 공개되지 않았습니다(2026-10-29 확인). GitHub 조직 `PaceVLA`에는 웹페이지 저장소 하나뿐이고, `iclr-pacevla/iclr-pacevla`는 익명 심사용 웹페이지 사본입니다. 그래서 코드 대조 대신 표·그림 재계산과, 관련 공개 코드(SAIL 지표 함수, VLASH 양자화 루프, GR00T RoboCasa 과제 표) 대조를 했습니다.

1. 재현에 필요한 정보 체크리스트

항목	논문에 있음	비고
모드 구성 (H_k, T_k)	✅	Table 8·9
손실 가중치·온도·최소 모드 확률·optimizer	✅	Table 7
양자화 집계 규칙	✅	부록 A, 식 5·6
벤치마크별 LVLM 프롬프트	✅	부록 E 6종
judge 임계값 δ	❌	"δ 이상이면 양자화 그룹"이라고만 적혀 있음
체크별 점수 가중치, 프롬프트의 "계산한 사실"	❌	부록 C.5에 "fixed weights", "computed facts"만 있음
과제별 k_p 선택 절차와 과제별 값	❌	{150, 300, 600} 중 과제별로 고른다고만 적혀 있음
decoder 구조(층 수, 폭), 라우터·judge 크기	❌	"lightweight MLP"
시뮬레이션 시행 수·시드	❌	표 수치에서 과제당 50회로 역산(C8)
실물 시행 수	✅	32회, 컨베이어는 5회 × 10개
학습 데이터 라벨(점수 y)	❌	공개되지 않음
코드·체크포인트	❌	미공개

2. 주장별 판정

#	주장 (출처)	확인	판정
C1	Table 5는 RoboCasa 24개 과제 평균 (캡션)	Table 1의 PnP 8 + Others 16 가중 평균이 SR 66.93, TPR 2.747, 풀링 ES 1.141 → Table 5의 66.9 / 2.75 / 1.14. 과제 구성은 GR00T 저장소 표로 확인	✅
C2	TPR(식 7)과 ES(식 8)의 정의	SR·TPR·t_max만으로 ES 36칸을 ±0.005 안에서 재현(`03` §1)	✅ 내부 일관
C3	"ES above 1× in all but one setting" (Sec. 4.2)	PaceVLA ES 9칸 중 1 미만은 RLDX-1 PnP 0.93 하나	✅
C4	"largely preserves the success rate"	기준 대비 SR 차이가 −3.5 ~ +2.7%p이고, 모두 유의하지 않음(PnP p = 0.48, Others 0.87, DexJoCo 0.85)	✅ (유지는 맞고, 향상은 아님)
C5	굵게 표시한 best (Table 1)	가속 방법 중 PaceVLA가 1위인 칸: SR 8/8, ES 7/8, **TPR 2/8**. TPR은 6칸에서 고정 양자화나 DemoSpeedUp이 더 높음	⚠️ "효율 개선"은 ES 기준일 때만 성립

#	주장 (출처)	확인	판정
C6	실물 수치(Sec. 4.2): Franka 37-45%, OpenArm 33-39% 단축, 고정 양자화 −28.2·−21.9%p, 컨베이어 3.6 → 6.3(1.75배), 최대 1.8배	37.2 / 40.7 / 45.0%, 33.3 / 39.3%, 28.2 / 21.9%p, 1.750배, 최대 시간비 1.82(Stack Cup)	✅
C7	Fig. 1(b) "averaged over six real-world tasks"	다섯 점이 6개 과제 단순 평균과 SR ±0.1%p, 속도 +0.02× 안에서 일치. 단, 축 이름 "over demonstration"과 캡션 "over the base VLA"가 다르고, 시간비(5개)와 처리량비(1개)를 섞어 평균함	✅ 수치 / ⚠️ 표기
C8	시행 수	실물 이진 과제 k/32, 부분 점수 과제 k/64, 컨베이어 k/50에 모두 맞음. 시뮬레이션은 과제당 50회(PnP 400, Others 800, DexJoCo 300)가 모든 값을 설명하는 최소 수	✅ (시뮬레이션 n은 역산)
C9	컨베이어 "improving success rate from 86% to 94%"	43/50 vs 47/50, Fisher p = 0.32	⚠️ 유의하지 않음
C10	추론 비용 1.05배, 약 2 ms (Table 6)	42.15 / 40.07 = 1.052, +2.08 ms	✅
C11	전용 decoder가 사후 양자화보다 정확 (Table 10)	보고값만 있음. MSE 평균 0.0218 vs 0.0294로 방향은 일관	- 검증 불가 (코드 없음)
C12	judge가 필요하다 (Table 5a)	judge 제거 시 TPR 2.75 → 2.01, ES 1.14 → 1.01. 단일 시드	⚠️ 경향
C13	고정 양자화 기준선 = VLASH [30]	VLASH `run.py` 는 `(step_count + 1) % action_quant_ratio == 0` 일 때만 행동을 전송. 식 5(블록 끝값)와 같은 동작	✅
C14	TPR은 SAIL을 따름	SAIL `throughput_regret_agg_fn` 과 식이 같음. 단위는 다름 (SAIL은 시뮬레이션 시간 초 × 60, PaceVLA는 행동 step 수 × 10^{-3})	✅
C15	컨베이어 과제 설명	본문 "라벨이 앞을 보게", 부록 C.1 "컨베이어에서 하역 테이블로", 지시문 "바코드 위로, 컨베이어 위에"	⚠️ 서로 불일치
C16	시간 양자화 이론 최대 속도	q = 2(LIBERO 1.6-1.78)이므로 명령 수 기준 최대 2배. 실물 1.50-1.82배, 시뮬레이션 성공 시행 step 비 1.18-1.45	참고
C17	v1 → v2 변경	본문 동일, Fig. 2 조건 화살표만 수정(첫 블록 Cross + Self → 두 블록의 Cross)	참고

3. 수치 재계산 요약

자세한 내용은 `03_analysis.md` 에 있습니다.

- **ES 분해(`03` §1.1):** t_max = 1500에서 기준 모델 "성공 1건당 step" 중 실패 몫이 PnP 78%, Others 65%, DexJoCo 55%입니다. ES는 성공률 유지에 크게 좌우됩니다.
- **t_max 민감도(`03` §1.2):** 실패 비용이 0이면 PnP에서 세 기준선이 모두 PaceVLA보다 빠릅니다. 순위가 뒤집히는 t_max는 14-115 step입니다.
- **추적 toy(`03` §2):** 추적 이득이 낮으면 적응형만 성공하고, 세밀 구간이 짧을수록 판정 지연에 민감합니다. 이득을 올리면 고정 양자화도 성공합니다. 그래서 과제별 이득 조정은 교란 변수입니다.
- **SO(3) toy(`03` §3):** q = 2에서 회전 벡터 단순 합과 SO(3) 합성의 차이는 스텝당 0.03 rad이면 0.02°입니다. 올바른 처리지만 성능을 가를 요소는 아닙니다.
- **실물(`03` §4):** PaceVLA의 시간은 고정 양자화의 0.92-1.10배입니다. 시간을 완전 성공 시행만으로 재서 실패 비용이 빠져 있습니다.

4. 구현 가능성 판단

4.1 공식 코드로 재현

불가능합니다. 코드, 체크포인트, quantizability 라벨이 공개되지 않았습니다. 논문에도 δ, 체크 가중치, 과제별 k_p, decoder 크기가 없어서, 코드가 나오기 전에는 같은 수치를 기대할 수 없습니다.

4.2 공개된 구성 요소로 다시 구현

구성 요소	공개 여부	난이도	메모
기반 VLA (GR00T N1.5, RLDX-1)	✅ Apache-2.0	-	GR00T 저장소 README(`c56f541` , 현재 N1.7 기준): 추론은 VRAM 16 GB 이상 GPU, 미세조정은 40 GB 이상 GPU 권장, CUDA 12.8과 flash-attn 설치
시간 양자화 목표 만들기 (식 5·6)	직접 구현	낮음	수십 줄. delta 행동은 제어기 좌표로 되돌린 뒤 합치고 다시 정규화
모드별 decoder (공유 DiT, 길이 T_k)	직접 구현	중간	GR00T action head의 행동 토큰 길이·위치 임베딩·마스크를 모드마다 다르게 다뤄야 함
라우터, judge, 식 2-4	직접 구현	낮음	MLP 두 개와 손실 세 개
quantizability 라벨	프롬프트만 공개	**높음**	과제마다 고정 양자화를 먼저 실행해 단서를 얻고, LVLM으로 데이터 전체를 라벨링해야 함. 가중치와 "계산한 사실"을 직접 정해야 함. 상용 API를 쓰면 데이터가 외부로 나가고 비용이 듦
제어기 이득 조정	절차 미공개	중간	과제별 k_p 탐색이 필요
벤치마크 (RoboCasa, LIBERO, DexJoCo)	✅	낮음	MuJoCo 기반. 시뮬레이션 자체는 CPU로 돌지만 정책 추론이 무거움
기준선 (VLASH, DemoSpeedup, SAIL)	✅ 코드 공개	중간	SAIL은 논문처럼 바꿔 재구현해야 같은 비교가 됨

4.3 이 맥에서 할 수 있는 것 (M4 Pro, CPU + Apple GPU(MPS), CUDA 없음)

단계	가능 여부	근거
표·그림 재계산, ES 분해, 지표 toy	✅ 완료	`table_check.py` , `quant_toy.py` (numpy, 10초 이내)
작은 flow matching chunk 정책으로 fine/quantized decoder + judge 재현(저차원 toy 과제)	조건부	torch 설치(MPS)가 필요해서 설치 승인을 받아야 함. 시간은 수십 분 예상
부록 E 프롬프트로 시연 몇 개를 라벨링해 Fig. 4 같은 점수 곡선 확인	조건부	LVLM API(데이터 외부 전송, 비용)와 RoboCasa 시연 일부 다운로드가 필요해서 승인이 필요함
GR00T N1.5 + PaceVLA 미세조정, RoboCasa 24개 평가	❌	CUDA GPU(40 GB 이상 권장)와 미공개 코드가 필요
실물 과제	❌	Franka, OpenArm, ALLEX 하드웨어

요약: 아이디어의 골격(모드별 decoder + 라우터 + 판정기)은 짧고 다시 구현하기 쉽습니다. 성능을 좌우할 부분은 두 가지입니다. 첫째는 판정 라벨을 만드는 과정으로, 과제별 사전 실행, LVLM 질문지 설계, 전체 라벨링이 필요합니다. 둘째는 과제별 제어기 이득 조정입니다. 둘 다 논문에 절차 일부만 공개되어 있습니다. 같은 수치를 원하면 코드 공개를 기다려야 하고, 아이디어를 시험하려면 작은 정책으로 "고정 양자화 + 이득 조정"과 "적응형"을 같은 이득에서 비교하는 것이 가장 싼 출발점입니다.

02. 선행연구와 계보

번호 [n]은 논문 참고문헌 목록의 순서(알파벳순, 본문 표에 쓰인 번호와 같음)입니다. arXiv ID는 논문 참고문헌에 적힌 값입니다. SAIL의 지표 함수, VLASH의 양자화 실행 루프, GR00T 저장소의 RoboCasa 과제 표는 코드를 직접 확인했습니다(§4). 나머지 논문은 따로 열어 검증하지 않았습니다.

1. 계보도

```
[action chunking 정책]
  ACT [36] → Diffusion·flow matching action head → GR00T N1.5 [5], RLDX-1 [18] (VLM + flow
matching DiT)
        │  chunk를 시연과 같은 시간 해상도로 예측
        ▼
[시연보다 빠르게 실행하기]
  ├─ RL로 다듬기        SpeedTuning [34]                              — 전체가 고르게 빨라지는 경향
  ├─ 데이터를 가속     DemoSpeedup [13] arXiv:2506.05064             — proxy 정책의 행동 엔트로피로 구
간별 다운샘플링
  ├─ 실행 시 조절      SAIL [2] arXiv:2506.11948                     — 운동 복잡도·그리퍼 사건으로 정해진
속도 사이 전환
  └─ 시간 양자화       Figure AI Helix 물류, VLASH [30] arXiv:2512.01031 — chunk를 고정 비율로 합침
        ▼
[모드를 고르는 라우팅]
  Switch Transformer [10] · MoH [15] · MoLe-VLA [35] (층 건너뛰기) · Mixture of Horizons [16]
arXiv:2511.19433 (여러 horizon을 gate로 융합)
        ▼
  ★ PaceVLA: 시간 해상도와 horizon이 다른 decoder 중 하나를 상태마다 고름 (LVLM 라벨로 학습한 judge + 라우
터)
```

차별점 요약: DemoSpeedup과 SAIL은 "어디서 빠르게 할지"를 행동 통계(엔트로피, 운동 복잡도, 그리퍼 사건)로 정합니다. 고정 양자화는 궤적 전체에 같은 비율을 씁니다. PaceVLA는 그 결정을 **장면을 보고 정책이 직접** 하게 하자는 것입니다. 다만 그 판단의 학습 신호는 사람(그리고 Claude)이 설계한 LVLM 질문지에서 옵니다(`01` §4.2-1).

2. 구성 요소별 출처

역할	출처	이 논문에서의 쓰임
기반 VLA	GR00T N1.5 [5], RLDX-1-IMG [18] arXiv:2605.03269	VLM 고정, action expert·decoder·라우터·judge 학습
flow matching	Lipman et al. [21] arXiv:2210.02747, DiT [28]	식 1
시간 양자화 규칙	VLASH [30]의 `action_quant_ratio` (코드 확인: 매 q번째 행동만 전송)	절대 행동은 블록 끝값(식 5), delta는 SO(3) 합성(식 6)
고정 양자화 기준선	VLASH [30]	기준 모델 출력에 q = 2
가속 기준선	DemoSpeedup [13], SAIL [2]	SAIL은 재구현(delta 행동, action scheduling 끔)
효율 지표 TPR	SAIL [2] `metric_fn.py`의 `throughput_regret_agg_fn`	식 7. SAIL은 시뮬레이션 시간(초)에 분당 단위를 쓰고, PaceVLA는 행동 step 수와 $\times 10^{-3}$ 단위를 씀
효율 지표 ES	Dex-Net의 MPPH [23]	식 8, 실패에 t_max를 매김

역할	출처	이 논문에서의 쓰임
라우터 균형 손실	Switch Transformer [10]	L_bal
실시간 chunking	RTC [6]	ALLEX 컨베이어에서만 비동기 추론
벤치마크	RoboCasa [26], LIBERO [22], DexJoCo [32] arXiv:2605.16257	RoboCasa 24개 = PnP 8 + Others 16 (GR00T 저장소 표로 확인)
라벨링 모델	Claude Opus 5(기준 설계), Gemini 3.8 Flash(전체 라벨링)	부록 C.5, E

3. 이 저장소의 다른 논문과의 관계

논문	관계
Chronos Policy (`chronospolicy/`)	둘 다 "정책이 시간을 어떻게 다루는가"의 문제입니다. Chronos는 관측과 행동의 **물리 시각**을 time-aware RoPE로 넣어 센서 주기·지연의 변화에 강하게 만듭니다. PaceVLA는 같은 시간 축에서 **명령 간격 자체**를 바꿉니다. PaceVLA의 양자화 모드는 행동 하나가 덮는 실제 시간을 2배로 바꾸는데, 정책은 그 차이를 decoder 번호로만 압니다. Chronos식으로 행동마다 실행 시각을 넣으면 decoder 여러 개 대신 연속적인 실행 간격을 표현할 수 있을 것입니다(내 해석).
AW-0 (`agentic_wam/`)	AW-0은 코드·subgoal 계획 스트림을 행동과 함께 생성합니다. PaceVLA의 judge가 보는 것도 "지금 어떤 국면인가(도달, 쥐기, 운반, 놓기)"입니다. 계획 스트림이 이미 있는 정책이라면 별도의 LVLM 라벨 없이 국면 정보를 judge 입력으로 쓸 수 있습니다(내 해석).
Hydra-0 (`hydra0/`)	행동 표현을 바꾸는 연구입니다. Hydra-0은 공간 표현(이미지 평면의 점 궤적)을 바꾸고, PaceVLA는 시간 해상도를 바꿉니다.
DexPolicy (`dexpolicy/`)	둘 다 "정밀한 국면과 여유 있는 국면은 다르게 다뤄야 한다"는 문제의식입니다. 다만 DexPolicy는 RL 학습 중의 탐색 잡음을 학습 step 함수로 줄이고, PaceVLA는 추론 시 실행 해상도를 상태 함수로 바꿉니다.
Contact-Grounded Policy (`contact_grounded_policy/`)	PaceVLA의 실패 분석(부록 D.3)은 "접촉 영역이 가려져서"라고 결론짓고 촉각 입력을 제안합니다. CGP처럼 촉각을 쓰는 정책에서는 접촉 시작 신호를 judge 입력으로 직접 쓸 수 있습니다.

4. 관련 GitHub 저장소 (2026-10-29 `gh api` 확인)

저장소	무엇인가	공식 여부	라이선스	★	마지막 push	확인 커밋
PaceVLA/pacevla.github.io	프로젝트 웹페이지(영상, 결과 표 이미지). **코드 없음**	공식	표기 없음(페이지 하단에 CC BY-SA 4.0 문구)	0	2026-10-28	`86424ec`
iclr-pacevla/iclr-pacevla	익명 심사용 웹페이지 사본(2026-09-24 생성). 현재 페이지와 저자 표기만 다름	공식으로 보임	표기 없음	0	2026-09-26	`4ef81f5`

저장소	무엇인가	공식 여부	라이선스	★	마지막 push	확인 커밋
NVIDIA/Isaac-GR00T	기반 VLA GR00T N1.5	구성요소 공식	Apache-2.0	8,280	2026-10-28	`c56f541` (`examples/robocasa/README.md`)
RLWRLD/RLDX-1	기반 VLA RLDX-1	구성요소 공식	Apache-2.0	355	2026-09-09	`cf67c31` (메타데이터만)
mit-han-lab/vlash	고정 양자화 기준선 [30]	공식으로 보임	Apache-2.0	514	2026-04-22	`22cbabf` (`vlash/run.py`)
lingxiao-guo/DemoSpeedup	기준선 DemoSpeedup	공식으로 보임	MIT	30	2025-09-10	`34bd43a` (메타데이터만)
nadunRanawaka1/SAIL	기준선 SAIL, TPR 정의	공식 (프로젝트 페이지 링크)	MIT	11	2026-07-19	`1440c16` (`metric_fn.py`)
Timsty1/MixtureOfHorizons	가장 가까운 관련 연구(ICML 2026)	공식으로 보임	표기 없음	68	2026-05-04	`5da3500` (README만)
Physical-Intelligence/real-time-chunking-kinetix	RTC	공식	MIT	631	2025-12-08	`9296f31` (메타데이터만)

저장소	무엇인가	공식 여부	라이선스	★	마지막 push	확인 커밋
DaivdYuan/SpeedTuning	RL 가속 관련 연구	공식으로 보임	MIT	17	2026-08-11	`b4a4cfc` (메타데이터만)
RoyZry98/MoLe-VLA-Pytorch	MoLe-VLA	확인 못함	MIT	70	2025-07-31	`8ae45f8` (메타데이터만)
robocasa/robocasa	벤치마크	공식	NOASSERTION	1,848	2026-10-28	`dc73a8c` (메타데이터만)
Lifelong-Robot-Learning/LIBERO	벤치마크	공식	MIT	2,488	2025-03-15	`8f1084e` (메타데이터만)
brave-eai/dexjoco	벤치마크 DexJoCo	공식으로 보임 (이름·시기 일치)	MIT	239	2026-10-15	`8d23b0f` (메타데이터만)

- `gh search repos PaceVLA` 결과는 위 두 웹페이지 저장소뿐이었습니다.
- 받은 파일은 `refs/` 에 있습니다(gitignore). `sail_metric_fn.py`, `gr00t_robocasa_README.md`, `vlash_README.md`, `vlash_run.py` 입니다.
- GR00T 저장소의 RoboCasa 평가 예시는 `--max-episode-steps 720 --n-action-steps 8` 입니다. 논문은 t_max = 1500을 씁니다(`03` §3).

5. 논문이 다루지 않았지만 관련 있는 연구 (내 판단)

주제	관련성	근거 수준
AWE: Automatic Waypoint Extraction (Shi, Sharma, Zhao, Finn, 2023)	시연을 "선형 보간으로 복원 가능한 최소 waypoint"로 압축합니다. 시간 해상도를 구간마다 다르게 두는 비균일 양자화의 데이터 쪽 버전입니다. PaceVLA의 LVLM 라벨 대신 재구성 오차 기준으로 "합쳐도 되는 구간"을 정하는 대안이 됩니다.	내 사전 지식. 서지 확인 필요
Options / macro-action (Sutton et al., 1999)	여러 step을 하나의 결정으로 묶는 시간 추상화의 고전 틀입니다. PaceVLA의 모드는 길이가 정해진 option입니다.	내 사전 지식

주제	관련성	근거 수준
사건 기반 제어·적응 샘플링 (제어 이론)	오차가 커질 때만 명령을 갱신하는 고전 기법입니다. judge 대신 추적 오차나 접촉 사건으로 세밀 모드를 켜는 규칙 기반 대조군이 될 수 있습니다.	내 해석
Bidirectional Decoding (BID)	chunk를 샘플링할 때 이전 chunk와의 일관성과 미래 예측을 함께 봐서 chunk 길이의 trade-off를 다룹니다.	내 사전 지식. 서지 확인 필요

한계: 5절의 서지 정보는 내 사전 지식에 근거합니다. 블로그나 git에 올리기 전에 원문을 확인해야 합니다.